

Active Reader — Tutor Pipeline Architecture Plan

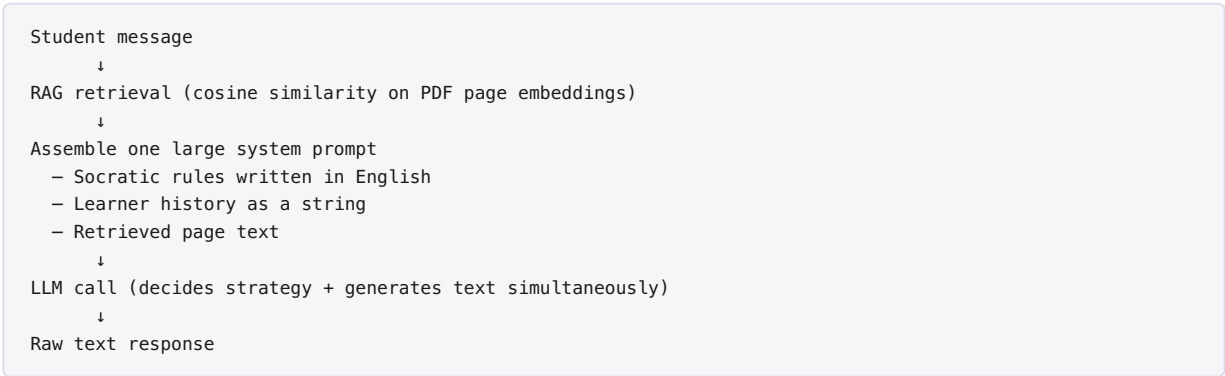
Stage 2: From Concept Graph to Adaptive Tutor *Technical Implementation Plan*

Abstract

The Active Reader tutor currently operates as a single stateless LLM endpoint: every student message triggers a RAG retrieval over raw PDF pages, a large prompt is assembled, and the LLM decides both the pedagogical strategy and the response text simultaneously. This document proposes splitting that monolith into a structured pipeline where deterministic code handles all decision logic and the LLM handles only natural language generation — a pattern directly borrowed from game AI architecture.

1. Current State

The entire tutor intelligence lives inside one endpoint: `POST /api/chat` (server.js, 676 lines).



Problems with this design:

Problem	Impact
LLM interprets "3+ exchanges" in English	Can miscount, give answers too early
No persistent student state	Every session starts from zero
Strategy and generation fused	Cannot test or log which decision was made
RAG on raw PDF pages	No concept-level grounding, no prerequisite awareness
No lesson structure	Tutor responds to whatever student asks, not a curriculum

2. The Core Principle: Split Decision from Generation

The key architectural change is borrowed directly from game AI:

Game AI:	Behavior Tree decides what to do Animator generates how it looks
Active Reader:	Structured code decides what to do LLM generates how to say it

The LLM never decides pedagogical strategy. It only generates natural language for a decision that code already made.

```
Structured code → WHAT to do and WHEN
LLM             → HOW to say it
```

3. The Three-Layer Architecture

Layer 1: Lesson State Machine

Tracks where the student is in a concept's lesson. States are compiled once from the concept graph node — no LLM calls at compile time.

```
States per concept:
HOOK      → present the study question
MOTIVATE  → why this concept matters
EXPLAIN   → core content
VISUAL    → linked figure
KEY_MOMENT → verbatim key passage from book
EXAMPLE   → worked example item
PRACTICE  → exercise item
├─ ATTEMPT
├─ HINT    (after N wrong attempts)
├─ RETRY
RECAP     → bullet summary
COMPLETE  → write to student model
```

Content for each state comes directly from the concept node fields:

```
concept.question.text → HOOK
concept.motivation.text → MOTIVATE
concept.content → EXPLAIN
concept.item_ids.figures → VISUAL
concept.key_passage.text → KEY_MOMENT
concept.item_ids.examples → EXAMPLE
concept.item_ids.exercises → PRACTICE
concept.recap_md → RECAP
```

Transitions are defined in a lookup table — not in prompt text:

```
const TRANSITIONS = {
  explain: { CONTINUE: 'visual', QUESTION: 'asking', BACK: 'motivate' },
  practice: { CORRECT: 'recap', WRONG: 'practice', STUCK: 'hint' },
  hint: { RETRY: 'practice', GIVE_UP: 'explain' },
}
```

Layer 2: Behavior Tree (Socratic Decision Logic)

Replaces the English rules in the current prompt. The tree runs in code and decides which LLM call to make. The LLM only executes the leaf action nodes.

```

Socratic Behavior Tree
|
├── Selector (first success wins)
|   |
|   ├── Sequence: student answered correctly
|   |   ├── Condition: last_score === 'correct'
|   |   └── Action: LLM('praise_and_advance', concept)
|   |
|   ├── Sequence: student is stuck (N wrong attempts)
|   |   ├── Condition: attempts >= 3
|   |   └── Action: LLM('give_hint', concept.key_passage)
|   |
|   ├── Sequence: scheduled check-in (every 3 exchanges)
|   |   ├── Condition: exchange_count % 3 === 0
|   |   └── Action: LLM('generate_mcq', concept)
|   |
|   └── Default: probe understanding
|       └── Action: LLM('socratic_question', concept.question)

```

The tree is testable: given a context object, you can verify exactly which branch fires without involving the LLM.

Layer 3: Student Model

A persistent record written after every interaction. Survives sessions. Drives the session scheduler.

```

StudentModel {
  concept_records: {
    [concept_id]: {
      status:      LOCKED | SEEN | PRACTICED | MASTERED,
      attempts:    int,
      last_correct: timestamp,
      retention_score: float,    // decays via forgetting curve
      next_due:    timestamp,    // spaced repetition cooldown
    }
  },
  struggle_profile: {
    weak_tags:    string[],    // concept tags with low retention
    stuck_concepts: string[],    // failed > N times
  },
  interactions: [
    { type, concept_id, content, timestamp }
  ]
}

```

4. Game Logic Patterns Applied

Five concrete patterns taken directly from game engine design:

4.1 Transition Table

States and events form a 2D lookup table. O(1) transition, no branching logic scattered across components.

4.2 Pre-compiled Lesson Lookup Table

All lesson plans compiled from concept nodes at startup. At runtime the state machine reads by index — no LLM calls during lesson execution.

4.3 Cooldown Timer → Spaced Repetition

After each correct recall, `next_due = now + interval[recall_count]`. The session scheduler polls due concepts exactly like a game polls ability cooldowns.

4.4 Dirty Flag → Prerequisite Propagation

When a concept is completed, set `student_model._dirty = true`. Unlock recalculation runs lazily when the curriculum map is next opened — not on every completion.

4.5 Interrupt State with Return Address

Student asking a question mid-lesson pushes current state to a stack and enters `ASKING`. On answer, pop the stack and return to wherever the lesson was.

```
// Student asks a question during EXPLAIN
push_interrupt('asking') // stack: ['explain']

// Question answered
pop_interrupt()           // returns to 'explain'
```

5. The Three LLM Roles

The LLM is called in three distinct modes — each with a specific template and grounding data:

Role	Endpoint	Input	Output
Scoring	<code>/api/score-answer</code>	student answer + correct answer	correct / partial / wrong + gap
Generation	<code>/api/chat</code> (split)	template type + concept node fields	1–2 natural language sentences
Retrieval	RAG → concept lookup	concept_id	concept node (no LLM needed)

The generation role receives one of four templates depending on the behavior tree decision:

```
give_hint(key_passage)      → "Here's a clue from the text: ..."
generate_mcq(concept)       → "Quick check – [question]? A) B) C)"
socratic_question(question) → "What do you think [question]?"
praise_and_advance(concept) → "Exactly. Now let's look at [next]..."
```

6. What Changes in `server.js`

The current `/api/chat` becomes an **event dispatcher**, not a reasoning engine:

```
// BEFORE: one big prompt, LLM decides everything
app.post('/api/chat', async (req, res) => {
  const prompt = build_giant_prompt(req);
  const reply = await llm(prompt);
  res.json({ reply });
});

// AFTER: code decides, LLM generates
app.post('/api/chat', async (req, res) => {
  const ctx = build_context(req);          // load concept node + student model
  const state = lesson_sm.dispatch(ctx);   // state machine: what state are we in?
  const action = behavior_tree(ctx);       // BT: what should we do?
  const reply = await llm(action.template, action.data); // LLM: generate text
  student_model.record(ctx, action, reply); // persist
  res.json({ reply, state });
});
```

7. Hierarchy Summary

Session Scheduler

reads: StudentModel.next_due per concept
outputs: next concept_id to study

↓

Book → Chapter → Concept (curriculum state machines)

each level: LOCKED / AVAILABLE / IN_PROGRESS / COMPLETE
dirty flag propagates unlocks lazily

↓

Lesson State Machine (per concept)

reads: LESSON_PLANS[concept_id] (pre-compiled lookup table)
transitions: TRANSITIONS table (event-driven)
interrupt stack: for questions, hints, pauses

↓

Behavior Tree (per exchange)

conditions: checked in code (attempts, exchange_count, last_score)
actions: select LLM template + grounding data

↓

LLM (generation only)

input: small focused template + concept node fields
output: 1-2 sentences of natural language

↓

Student Model (persistent)

written: after every LLM response
read: by session scheduler at next session start

8. Build Order

Step	Task	Depends On
1	Lesson plan compiler: concept node → <code>lesson_plans/*.json</code>	Concept graph (done)
2	Transition table + lesson state machine	Lesson plans
3	Student model schema + file persistence	Nothing
4	Behavior tree (replaces prompt rules in <code>server.js</code>)	Student model
5	Wire concept lookup to replace RAG in <code>/api/chat</code>	Lesson SM + BT
6	Session scheduler (spaced repetition)	Student model
7	Book / chapter level state machines	Lesson SM + Student model

Step 1 is the immediate next action — it directly bridges the completed concept graph pipeline to the tutor runtime.